To make it possible to submit orders successfully, we'll need to make Order Service call Catalog Service to check the book's availability and fetch the necessary information for processing the order. That's the focus of the next section. Before moving on, stop the application with Ctrl-C.

8.3 Reactive clients with Spring WebClient

In a cloud native system, applications can interact in different ways. This section focuses on the request/response interactions over HTTP that you'll establish between Order Service and Catalog Service. In this kind of interaction, the client making the request expects to receive a response. In an imperative application, that would translate into a thread blocking until a response is returned. Instead, in a reactive application, we can use resources more efficiently so that no thread will wait for a response, freeing up resources to deal with other processing.

The Spring Framework comes bundled with two clients that perform HTTP requests: `RestTemplate` and `WebClient`. `RestTemplate` is the original Spring REST client that allows blocking HTTP request/response interactions based on a template method API. Since Spring Framework 5.0, it's in maintenance mode and practically deprecated. It's still widely used, but it will not get any new functionality in future releases.

`WebClient` is the modern alternative to `RestTemplate`. It provides blocking and non-blocking I/O, making it the perfect candidate for both imperative and reactive applications. It can be operated through a functional-style, fluent API that lets you configure any aspect of the HTTP interaction.

This section will teach you how to use `WebClient` to establish non-blocking request/response interactions. I will also explain how to make your